

MAE 4100/5100 Numerical Methods for Fluids

Course Project II (FDM-based CFD Solver)

Instructor: Dr. Jian-Xun Wang

Associate Professor, Department of Mechanical and Aerospace Engineering

Note: Certain phases would be compulsory only for students of MAE 5100, and bonus for students of MAE 4100.

Phase I (Main solver implementation), Due: Apr 28th

Phase II (Vectorize the code), Due: May 12th

Phase III (V&V, parametric study, and final report), Due: May 12th

1 Phase I: Density-based CFD Solver Implementation (40 pts / 30 pts)

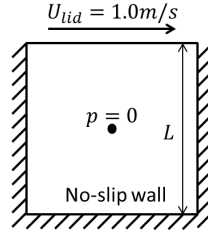


Figure 1: Geometry of the lid-driven cavity

Write a CFD code to solve for the flow in a square lid-driven cavity with the top wall moving at velocity U_{lid} . Use the 2D incompressible Navier-Stokes equations with time derivative preconditioning (assuming constant temperature and viscosity μ) given by

$$\frac{1}{\beta^2} \frac{\partial p}{\partial t} + \rho \frac{\partial u}{\partial x} + \rho \frac{\partial v}{\partial y} = S$$

$$\rho \frac{\partial u}{\partial t} + \rho u \frac{\partial u}{\partial x} + \rho v \frac{\partial u}{\partial y} + \frac{\partial p}{\partial x} = \mu \frac{\partial^2 u}{\partial x^2} + \mu \frac{\partial^2 u}{\partial y^2}$$

$$\rho \frac{\partial v}{\partial t} + \rho u \frac{\partial v}{\partial x} + \rho v \frac{\partial v}{\partial y} + \frac{\partial p}{\partial y} = \mu \frac{\partial^2 v}{\partial x^2} + \mu \frac{\partial^2 v}{\partial y^2}$$

where S is an artificial viscosity term given by

$$S = -\frac{|\lambda_x|_{max} C^{(4)} \Delta x^3}{\beta^2} \frac{\partial^4 p}{\partial x^4} - \frac{|\lambda_y|_{max} C^{(4)} \Delta y^3}{\beta^2} \frac{\partial^4 p}{\partial y^4}$$

where $|\lambda_x|_{max}$ is the magnitude of the largest eigenvalues in (x, t) space, $|\lambda_y|_{max}$ is the magnitude of the largest eigenvalues in (y, t) space, and $C^{(4)}$ is a constant that generally lies in the range,

$$\frac{1}{128} \leq C^{(4)} \leq \frac{1}{16}.$$

In the above equation, β is the time-derivative preconditioning term given by,

$$\beta^2 = \max(u^2 + v^2, \kappa U_{lid}^2)$$

where κ can range from 0.001 to 0.9.

You will be provided with a template code in Python (Turn off the Method of Manufacture Solution (MMS), i.e., $imms = 0$ in Phase I). Use the simple explicit relaxation method (i.e., a point Jacobi) with second-order accurate central differences to advance the discrete equations in pseudo-time until you reach the steady-state solution. Please also implement an explicit (point) symmetric Gauss-Seidel (SGS) scheme. Monitor iterative convergence using the steady-state iterative residuals (i.e., the steady portion of your discretization evaluated all at the same time level). A relative iterative convergence (i.e., the ratio of iterative residuals to initial iterative residuals at step 1) of at least 8 orders of magnitude is recommended. You can use the residual $R_{i,j}$ with the form of

$$-R_{i,j}^n = (u_{i,j}^{n+1} - u_{i,j}^n) / \Delta t$$

Use the stability criteria we discussed in class to determine the time step; you may choose local or global time stepping, but local time stepping is recommended (i.e., take the largest allowable time step at each node).

Run a baseline case at Reynolds number of 100 with the following conditions:

$$\begin{aligned} \text{Re} &= \frac{\rho U_{lid} L}{\mu} = 100 \\ 0 &\leq x \leq 0.05m \\ 0 &\leq y \leq 0.05m \\ L &= 0.05m \\ \rho &= 1.0 \text{ kg/m}^3 \\ U_{lid} &= 1.0 \text{ m/s} \end{aligned}$$

Baseline mesh starts from a coarse mesh (65×65 nodes, starting with $i = j = 1$ at the lower left-hand corner where $x = y = 0$). The code is set up to perform pressure re-scaling to enforce a reference pressure at the center of the cavity, with the reference pressure $p_{ref} = 0.8013338444662 \text{ N/m}^2$

Set up a simulation with the exact the same setting in OpenFOAM. Compared the simulated solutions obtained from your Python solver and the OpenFOAM. Please plot the contours comparison, and plot the u-velocity profile, v-velocity profile, and pressure profile on the vertical centerline ($x = 0.5L, y \in [0, 1]$), against OpenFOAM results.

Tasks in Phase I:

- Read the entire template code (main_solver.py, req_functions.py, utils.py) and understand the code structure. You will need to run main_solver.py, and implement subroutines in req_functions.py.
- Implement the following subroutines in req_functions.py to enable the 2D cavity solver:
 - function bndry()
 - function [dtmin] = compute_time_step(dtmin)
 - function Compute_Artificial_Viscosity()
 - function point_Jacobi()
 - function SGS_forward_sweep()
 - function SGS_backward_sweep()
 - function [res, resinit, conv] = check_iterative_convergence(n, res, resinit, ninit, rtime, dtmin)
- Plot steady-state u, v, p contours
- Plot velocity, and pressure profiles on the vertical centerline,
- Submit a PDF document with these plots (detailed report is not required at this stage) and your code through Canvas by due of phase I.
- For MAE 5100 students, you need complete additional phase I tasks related to pressure-based solver, see Sec 4.

2 Phase II: Vectorize the code (15+5 pts)

Vectorize the Python code to speed up the solver. This will help you in Phase III solution verification.

Tasks in Phase II:

- The following functions will be required to vectorize -
 - compute_time_step
 - Compute_Artificial_Viscosity
 - bndry
 - bndrymms
 - point_Jacobi
 - check_iterative_convergence
 - (BONUS) SGS_forward_sweep and SGS_backward_sweep

- Validate vectorized code: Plot velocity, and pressure profiles on the vertical centerline, and compare with serial code implementation
- Report computational time required for serial and vectorize solver in a table.
- (BONUS) **Parallelizing Symmetric Gauss-Seidel:** Standard row-major iteration of the Gauss-Seidel solver creates strict spatial dependencies. Executing a forward sweep at node (i, j) requires updated state data from adjacent nodes $(i - 1, j)$ and $(i, j - 1)$, precluding conventional loop parallelization. Identify node sets exhibiting zero mutual dependence and formulate a parallel update strategy. Credit awarded only if the sufficient explanation of the algorithm is accompanied by accurate implementation within `SGS_forward_sweep_acc()` and `SGS_backward_sweep_acc()`, and `solve_PPE_SGS_acc()` (Only for MAE 5100 students - Full bonus credit will be provided for MAE 4100 students if first 2 functions are implemented).

3 Phase III: CFD Code/Solution Verification and Project Report (45 pts / 35 pts)

Note: Please remember that the simulation on the finest mesh may take you several hours or a day to converge to the steady state, so it is recommended to use vectorized code and please start early!

The Python code template has the capability to run manufactured solutions (by setting the `imms` input flag to one instead of zero).

Tasks in Phase III:

- Perform a code verification study by computing the discretization error norms for the manufactured solution case on a series of systematically-refined meshes. Use a Reynolds number of

Discretize and solve on multiple meshes (Uniform)		
Mesh Name	Mesh Nodes	Grid Spacing, h
Mesh 1	129 * 129	1
Mesh 2	65 * 65	2
Mesh 3	33 * 33	4
Mesh 4	17 * 17	8
Mesh 5	9 * 9	16

Figure 2: Systematically refined meshes

10, compute the observed order of accuracy, and show that it approaches second-order with mesh refinement. **Note:** to implement function `Discretization_Error_Norms(rL1norm, rL2norm, rLinfnorm)`

- Run Reynolds numbers of 10, 100, 500 and, 1000 analyze the flow. (note, these cases with higher Reynolds number may require finer meshes to be stable.) You will be provided with numerical solutions from the literature for comparison purpose.
- Study the effect of mesh size

- Study the effect of κ parameter in the time derivative preconditioning (recommend on at least 65 by 65 nodes).
- Study the effect of $C^{(4)}$ constant in the artificial viscosity (recommend on at least 65 by 65 nodes).
- Comparison with literature: please compare your solution against the results from the literature provided (Plot the profile comparisons).
- Compare cases with different geometries (1) aspect ratio is 2:1 in x and y direction, i.e, width and height of the cavity are 0.1 and 0.05, respectively; (2) aspect ratio is 1:2 in x and y directions, i.e., width and height of cavity are 0.05 and 0.1, respective.
- (Bonus 2pts) Comparison with OpenFOAM: please use OpenFOAM to conduct the same cavity flow simulation, and compare the results from your own code with the results from the OpenFOAM (Re = 10, 50, and 100).
- For MAE 5100 students, you need complete additional phase III tasks related to pressure-based solver, see Sec 4.
- Submit a final project report, including sections:
 - Theory (including subsections like governing equations, boundary conditions, discretization, stability, artificial viscosity, artificial compressibility, Jacobi/SGS algorithm)
 - Iterative convergence analysis for Jacobi and SGS (plot and analyze convergence history for u, v p..)
 - Code verification, observed order of accuracy
 - discretization error estimation
 - Flow analysis (u, v, p) for different Reynolds number (10, 100, 500, 1000), please give contour and/or streamline plots and provide your analysis.
 - Parameter study, including effects of κ , $C^{(4)}$ and mesh size (given one Reynolds number)
 - Compare cases with different geometries (1) aspect ratio is 2:1 in x and y direction, i.e, width and height of the cavity are 0.1 and 0.05, respectively; (2) aspect ratio is 1:2 in x and y directions, i.e., width and height of cavity are 0.05 and 0.1, respective. Show comparison plots in terms of contours and profiles.
 - Comparison with literature results (and OpenFOAM* for bonus) – please plot contours, velocity, pressure profiles on vertical and horizontal center lines.
 - **Advanced Solver Analysis:** Discuss the performance, iterative scaling, and contour differences between your artificial compressibility (Phase I) solver and your newly implemented decoupled fractional step solver.
 - Append your Python code as a separate file (please make sure it can run and generate the results in the report) and also including your coded subroutines in as an appendix in your report.

4 (MAE 5100 Only): Pressure-based Solver - Fractional Step Method (20 pts)

Note: This section is 10 pts BONUS for undergraduates (MAE 4100), while for graduates (MAE 5100) it is compulsory.

MAE 5100 students are required to implement the decoupled Fractional Step Method (Chorin Projection). This algorithm resolves the incompressible Navier-Stokes equations without reliance on non-physical artificial compressibility parameters ($\kappa, C^{(4)}$).

The fractional step method decouples the velocity and pressure fields into a strict three-step sequence.

1. Predictor Step Calculate the intermediate divergent velocity field $\vec{u}^*$ explicitly from the momentum equations, omitting the pressure gradient. The discrete implementation utilizes second-order central differencing for convective and viscous terms, and incorporates local time stepping ($\Delta t_{i,j}$) and analytical momentum source terms (S_x, S_y) for Method of Manufactured Solutions (MMS) verification.

$$u_{i,j}^* = u_{i,j}^n + \Delta t_{i,j} \left[-u_{i,j}^n \frac{u_{i+1,j}^n - u_{i-1,j}^n}{2\Delta x} - v_{i,j}^n \frac{u_{i,j+1}^n - u_{i,j-1}^n}{2\Delta y} + \nu (\nabla_{discrete}^2 u^n) + \frac{S_{x,i,j}}{\rho} \right]$$

$$v_{i,j}^* = v_{i,j}^n + \Delta t_{i,j} \left[-u_{i,j}^n \frac{v_{i+1,j}^n - v_{i-1,j}^n}{2\Delta x} - v_{i,j}^n \frac{v_{i,j+1}^n - v_{i,j-1}^n}{2\Delta y} + \nu (\nabla_{discrete}^2 v^n) + \frac{S_{y,i,j}}{\rho} \right]$$

2. Pressure Poisson Equation (PPE) Solve the elliptic Poisson equation iteratively to enforce the divergence-free constraint. The right-hand side source term integrates the intermediate velocity divergence and the MMS mass source term (S_{mass}).

$$\nabla^2 p^{n+1} = \frac{\rho}{\Delta t_{i,j}} \left(\nabla \cdot \vec{u}^* - \frac{S_{mass,i,j}}{\rho} \right)$$

The discrete Laplacian is resolved using point Jacobi, Symmetric Gauss-Seidel (SGS). Zero-gradient Neumann boundary conditions ($p_{wall} = p_{interior}$) are strictly enforced at all boundaries per iteration. Convergence is achieved when the maximum absolute pressure differential drops below the designated tolerance threshold.

3. Corrector Step Project the intermediate velocity field onto a divergence-free domain utilizing the converged pressure field. The update employs second-order central differencing for the pressure gradients.

$$u_{i,j}^{n+1} = u_{i,j}^* - \frac{\Delta t_{i,j}}{\rho} \frac{p_{i+1,j}^{n+1} - p_{i-1,j}^{n+1}}{2\Delta x}$$

$$v_{i,j}^{n+1} = v_{i,j}^* - \frac{\Delta t_{i,j}}{\rho} \frac{p_{i,j+1}^{n+1} - p_{i,j-1}^{n+1}}{2\Delta y}$$

Tasks in Phase I:

- **Subroutine Implementation:** Code the serial and vectorized decoupled logic in `req_functions.py`:
 - `compute_intermediate_velocity()`

- `solve_PPE_Jacobi()`
- `solve_PPE_SGS()`
- (BONUS) `solve_PPE_SGS_acc()` (refer to Phase II bonus task for more details.)
- `correct_velocity()`

- **Solver Integration:** Toggle `solver_method = 'fractional_step'` in the `main_solver.py` sequence.

Tasks in Phase II/III:

- **MMS Code Verification:** Execute the Method of Manufactured Solutions (`imms = 1`) using the fractional step solver. Compute L1, L2, and Linf discretization error norms across multiple grids. Prove retention of formal 2nd-order spatial accuracy. (Note: you will observe 1st-order spatial accuracy for the pressure, particularly for the Linf discretization error norm. Explain why is this the case? HINT: Look at the Pressure Poission Equation implementation.)
- **Odd-Even Decoupling Test:** Execute the baseline cavity simulation (41×41 or 81×81) at $Re = 100$. Check whether artificial viscosity has any influence over the results. Formulate the theoretical basis for why the decoupled projection method suppresses non-physical checkerboarding without artificial stabilization.
- **Algorithm Performance Analysis:** Generate an iterative residual convergence plot overlaying the coupled formulation (density-based solver) against the decoupled formulation (pressure-based solver). Contrast runtime metrics and define the physical meaning underlying the differing convergence scales.